

MPLADS AI Command Center

Master Project Technical Knowledge Base & Complete Reference Guide

Project Documentation

September 2026

Contents

About This Document	3
Module 1: Problem Statement & Solution Overview	4
Executive Overview & Real-World Context	4
The Core Governance Challenge	4
Pitch Versions for Stakeholders	4
Exactly What Was Built — Solution Architecture	6
Feature-by-Feature Breakdown	7
Module 2: Complete System Architecture	8
Layered Architecture	8
Component Responsibilities	8
Module 3: Data Pipeline & Dataset Lineage	10
Data Lineage & Lifecycle	10
Dataset Statistics & Coverage	10
Module 4: Live Scraper Engine Deep Dive	11
Architecture (scraper/)	11
Change Detection Workflow	12
Module 5: Database Architecture & ER Design	13
Dual-Database & Analytical Parquet Architecture	13
Core Database Tables	13
Module 6: Analytical & Machine Learning Core Models	14
Model 1: Cost Anomaly Detector (ml_models/cost_anomaly/)	15
Model 2: Duplicate Work Detector (ml_models/duplicate_work/)	15
Model 3: Fund & Expenditure Anomaly Detector (ml_models/fund_expenditure_anomaly/) 1	
Phase 5: Delay & Statutory SLA Rule Engine (rule_engines/delay/)	15
Module 7: Feature Engineering Registry	16
Pipeline (feature_engineering/)	16
Module 8: Statutory SLA Rule Engine	17
Rule Engine (rule_engines/delay/)	17
Module 9: FastAPI Backend Application Gateway	18
Architecture (api/)	18
Module 10: Authentication, RBAC & Security Architecture	19
Cryptographic Security Stack	19
Timing Attack Defense	19
Server-Side RBAC & Jurisdictional Scoping	20
IDOR Protection Strategy	21

Module 11: Frontend Command-Center UI/UX	22
React 18 Single-Page Application (frontend/)	22
Module 12: Subho AI Governance Chatbot	23
Architecture (api/routers/chat.py)	23
Module 13: Automated Test Suite & QA	24
Architecture (tests/)	24
Module 14: Performance Optimization & Deployment	26
Performance Optimizations	26
Deployment Workflow	26
Module 15: Complete Repository File Map	27
Module 16: Architecture & Data Flow Diagrams	29
High-Level System Flow	29
Database Relationship Overview	30
Module 17: Viva Questions & Expert Answers	31
Q1: Why split anomaly detection into 4 separate models instead of one composite risk score?	31
Q2: How does Model 1 guarantee zero data leakage?	31
Q3: How do you prevent username enumeration and timing attacks on login?	31
Q4: Why is District Officer scoping based on (state, district) tuples rather than district names alone?	31
Module 18: Technology Selection Justifications	32
Module 19: Evaluator & Reviewer Q&A	33
What makes this project innovative compared to a standard government dashboard?	33
How does the system handle false positives in Duplicate Work Detection?	33
How scalable is the architecture?	33
Module 20: One-Page Master Reference	34
Closing Note	35

About This Document

This book is a complete, consolidated technical reference for the **MPLADS AI Command Center** — an AI-powered governance and analytics platform built to monitor the Members of Parliament Local Area Development Scheme (MPLADS) across India.

It brings together the problem statement, system architecture, data pipeline, all four analytical engines, database design, security model, frontend design, testing strategy, deployment guide, repository map, architecture diagrams, viva/interview questions, technology justifications, and a one-page cheat sheet — all in a single shareable document.

Project Scope at a Glance

Metric	Value
Master Works Tracked	190,942
Expenditure Vouchers	109,311
Total Sanctioned Outlay	₹10,211.49 Crore
Total Disbursed Capital	₹10,166.10 Crore
Fund Utilization Rate	99.56%
States / Union Territories Covered	36
Districts Covered	773
Automated Tests	117 / 117 Passing (100%)

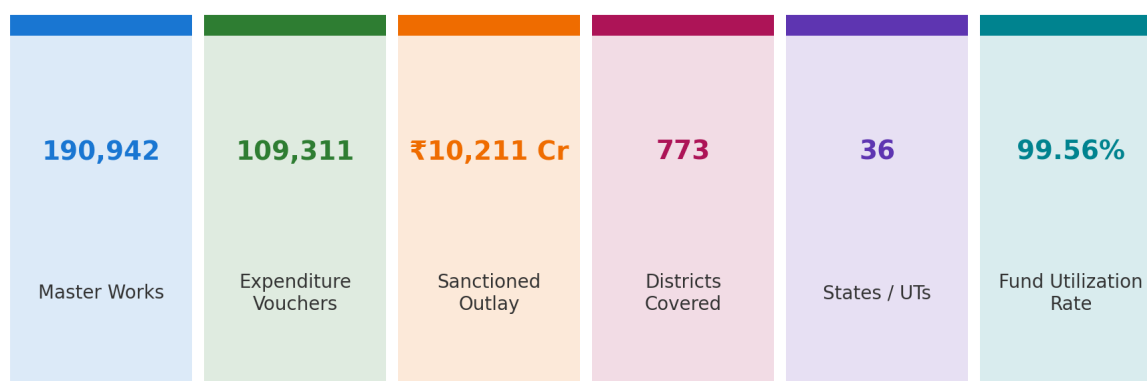


Figure 1: Project scope at a glance

Module 1: Problem Statement & Solution Overview

Executive Overview & Real-World Context

The **Members of Parliament Local Area Development Scheme (MPLADS)** is a flagship Central Sector Scheme fully funded by the Government of India. Under this scheme, each Member of Parliament (MP) has the entitlement to recommend developmental works in their constituency costing up to **₹5 Crore per annum**, focusing primarily on creating durable community assets such as drinking water systems, primary health centers, rural roads, public libraries, school infrastructure, and sanitation facilities.

With **788 MPs** across both houses of Parliament (Lok Sabha and Rajya Sabha), over **₹3,900 Crore** is allocated annually for public assets across **36 States/UTs** and **773 Districts**. Across multiple parliamentary terms, this results in over **190,000 active and historical works** worth over **₹10,211 Crore**.

The Core Governance Challenge

Monitoring government asset creation across 773 districts presents massive systemic hurdles:

1. **Massive Scale & Volume** — Manually reviewing 190,942 individual work recommendations, sanction orders, and 109,311 payment vouchers across 773 districts is physically impossible for human vigilance officers.
2. **Cost Overruns & Inflated Estimates** — Due to fragmented local tenders, sanction costs for identical works (e.g. *Installation of 50W Solar Street Light*) vary by 10x to 25x across adjacent districts or states without triggering red flags.
3. **Duplicate Work Sanctions & Double Billing** — Identical or overlapping physical projects (e.g., *Community Hall Construction*) are frequently sanctioned and billed multiple times under slightly altered names across consecutive years, adjacent boundaries, or different MPs.
4. **Financial Bottlenecks & Voucher Discrepancies** — Funds sit unutilized for years after sanction, or completed projects reflect 0 expenditure vouchers on portal records due to administrative reporting lag.
5. **Severe Statutory SLA Delays** — Over 51% of recommended works breach Para 3.12 of the official MPLADS Guidelines, which mandates that District Authorities accord sanction within **75 days** of receiving an MP recommendation.

Pitch Versions for Stakeholders

One-Line Problem Statement

“Manual oversight fails at scale across 190,000+ distributed public works, enabling cost inflation, duplicate billing, dormant funds, and severe statutory SLA breaches.”

30-Second Elevator Pitch

“Under the MPLAD Scheme, MPs allocate ₹5 Crore annually for local infrastructure across 773 districts. Because monitoring nearly 200,000 works manually is impossible, millions of rupees are lost to inflated estimates, duplicate project billing, dormant allocations, and years of bureaucratic delay. We built the MPLADS AI Command Center — an end-to-end analytics platform powered by 4 independent ML and rule engines that automatically audits every work for cost anomalies, semantic duplicates, fund irregularities, and statutory delay SLA breaches in real-time.”

2-Minute Presentation Pitch

“India’s MPLAD Scheme funds thousands of vital community assets every year. However, central and state authorities face a massive data dark-spot: 190,942 works worth over ₹10,200 Crore are recorded across fragmented portal dumps. Human vigilance teams cannot detect when a school hall in Bihar is sanctioned at 15 times the median state cost, or when two identical road repair projects are billed under different titles in the same district.

Our solution, the MPLADS AI Command Center, transforms raw portal dumps into actionable governance intelligence. First, an automated ingestion scraper normalizes raw records into a canonical dataset. Next, four dedicated analytical engines evaluate each work independently:

1. A Hierarchical Peer-Grouped Isolation Forest detects inflated cost estimates at sanction time with zero data leakage.
2. A Transformer-based Sentence-BERT model combined with structural proximity signals detects duplicate work candidate pairs.
3. A Multivariate Expenditure Isolation Forest flags voucher concentration anomalies and status-expenditure mismatches.
4. A Statutory Rule Engine enforces the official 75-day sanction SLA and 365-day execution guidelines.

Finally, an enterprise FastAPI gateway and React 18 command-center dashboard present role-scoped risk dossiers to Central Ministry, State, District, and MP stakeholders, complete with an AI assistant for natural-language inquiry.”

Detailed Technical Problem Statement

“To design, implement, and validate an automated, scalable data-processing and machine-learning governance platform capable of ingesting heterogeneous administrative records of the MPLADS scheme, normalizing currency, date, and text fields into a canonical schema of 190,942 works, engineering domain-specific feature registries, executing four strictly independent anomaly and compliance models (Cost Anomaly Isolation Forest, Sentence-Transformer Duplicate Matcher, Expenditure Cohort Isolation Forest, and Statutory SLA Rule Engine), storing results

in a cloud PostgreSQL database with strict Row Level Security, and exposing server-side RBAC-scoped REST APIs to a responsive React command-center interface.”

Exactly What Was Built — Solution Architecture

The **MPLADS AI Command Center** is an integrated governance analytics platform consisting of:

- **Live Data Scraper** (scraper/) — Automated crawler utilizing Scraping and Async HTTP to ingest raw portal dumps, compute SHA-256 content hashes, track change logs, and log ingestion runs.
- **Canonical Feature Pipeline** (data_pipeline/, feature_engineering/) — Cleaners that standardize Indian currency (₹), ISO dates, and tab control characters, unifying 190,942 works and 109,311 vouchers.
- **4 Independent Analytical Core Engines:**
 - **Model 1 — Cost Anomaly Detector** (ml_models/cost_anomaly/): Peer-grouped Isolation Forest benchmarking cost estimates against state/national work medians.
 - **Model 2 — Duplicate Work Detector** (ml_models/duplicate_work/): all-MiniLM-L6-v2 dense embeddings (384-d) combined with structural proximity scoring across a 90-day candidate blocking window.
 - **Model 3 — Fund Anomaly Detector** (ml_models/fund_expenditure_anomaly/): Isolation Forest on active spenders + Herfindahl-Hirschman Index (HHI) payment concentration + deterministic status mismatch rules.
 - **Phase 5 — Delay Rule Engine** (rule_engines/delay/): Statutory SLA rule engine enforcing 75-day sanction limits and 365-day work completion windows against a fixed reference date (2026-09-05).
- **Supabase PostgreSQL 17.6 Relational Core** (database/) — Cloud database enforcing zero orphaned foreign keys and Row Level Security.
- **FastAPI Application Gateway** (api/) — OAuth2 Bearer JWT authentication, Bcrypt 12-round hashing with constant-time dummy timing attack mitigation, and server-side SQL predicate injection for 4 governance tiers (MINISTRY, STATE_OFFICER, DISTRICT_OFFICER, MP).
- **React 18 Command Center SPA** (frontend/) — Vite, TypeScript, Tailwind CSS, Recharts, interactive district/MP summaries, work dossiers, and a “Subho” AI chatbot powered by a Groq LLM fallback chain.

Feature	What	Why	How	Output	Who Uses It
---------	------	-----	-----	--------	-------------

Feature-by-Feature Breakdown

Feature	What	Why	How	Output	Who Uses It
Cost Anomaly Audit	Detects inflated cost estimates	Prevents tender price padding	Peer Isolation Forest	Calibrated score [0,1], severity (HIGH/MED/L)	District Officers & Ministry
Duplicate Work Matcher	Flags double-billed projects	Stops double funding	Sentence-BERT + structural scoring	Candidate pairs with similarity score & reasons	Vigilance Officers & State Authorities
Fund Anomaly Monitor	Identifies payment irregularities	Prevents voucher fraud & dormant funds	Cohort Isolation Forest & HHI	Audit category, HHI score, anomaly reasons	Ministry & District Planning Officers
Statutory Delay Tracker	Measures SLA compliance	Enforces legal timelines	Rule Engine (Para 3.12)	Delay score, primary delay type, overdue days	MPs & State Officers
Subho AI Chatbot	Conversation query interface	Instant natural-language insights	Groq LLM + context injection	Natural language response with suggestions	All stakeholder roles

Module 2: Complete System Architecture

Layered Architecture

The MPLADS AI Command Center is designed as a decoupled, multi-tiered enterprise architecture:

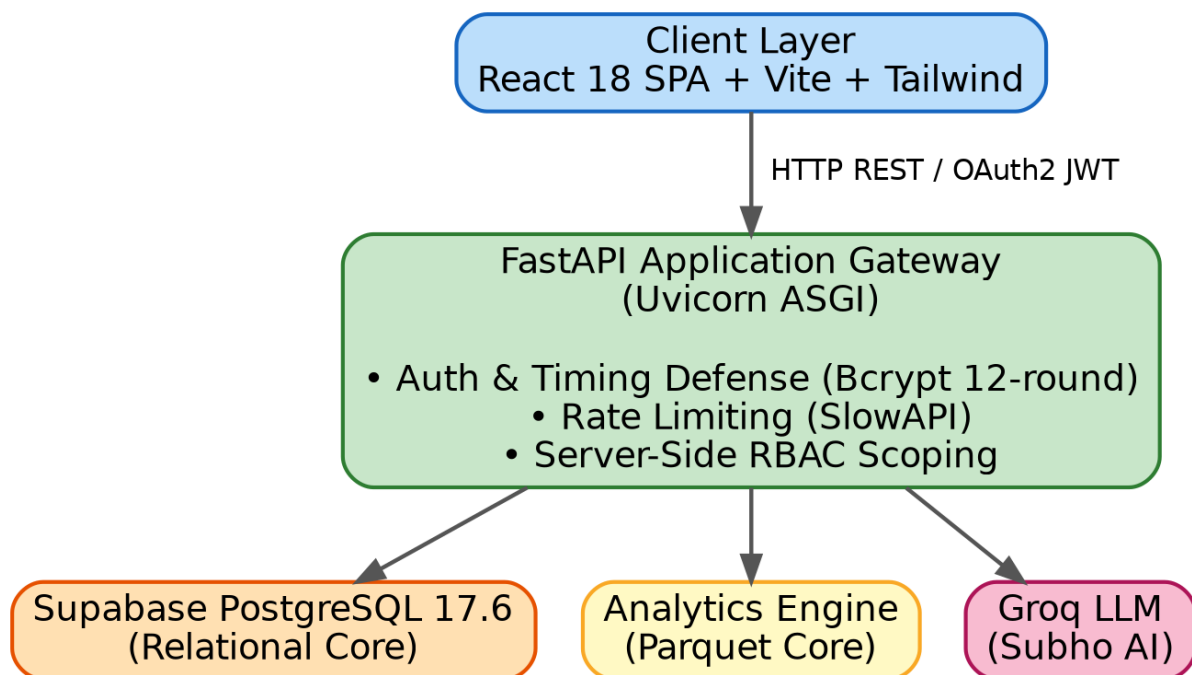


Figure 2: High-level layered architecture

Component Responsibilities

1. Data Ingestion & Scraper Tier (scraper/, data_pipeline/)

- Scraper Spider (scraper/spider.py): Crawls government portal endpoints asynchronously, storing source snapshots.
- SHA-256 Hash Change Detection (scraper/change_detection.py): Computes SHA-256 content hashes of raw responses to track new, updated, and unchanged records.
- Pipeline Cleaners (data_pipeline/cleaners.py): Sanitizes raw strings, strips tab characters, parses Indian currency values, and normalizes mixed date formats to ISO YYYY-MM-DD.

2. Analytical Core Tier (ml_models/, rule_engines/)

Executes 4 strictly independent analytical engines: Cost Anomaly Isolation Forest, Duplicate Work Sentence Transformer, Fund Expenditure Isolation Forest, and the Statutory SLA Delay Rule Engine.

3. Database Tier (database/)

- Supabase PostgreSQL 17.6: primary production cloud relational database storing the master catalog works, model outputs, work_expenditures, and users.
- SQLite (mplads_master.db): local development and offline testing fallback.
- Parquet storage (data/model_outputs/): high-performance columnar storage for fast offline analytics and training.

4. API & Application Gateway Tier (api/)

- FastAPI ASGI application handling REST endpoints, OpenAPI documentation, and SPA routing.
- Security & RBAC (api/auth/): validates JWT signatures, re-verifies user active status in PostgreSQL, and injects jurisdictional WHERE predicates based on caller role.

5. Presentation & User Experience Tier (frontend/)

React 18 SPA built with Vite and TypeScript, featuring a dark command-center UI, responsive navigation, dynamic Recharts visualizations, interactive maps, and the Subho AI chatbot widget.

Module 3: Data Pipeline & Dataset Lineage

Data Lineage & Lifecycle

Data flows in one direction, end to end, from raw government portal dumps to the dashboards stakeholders see:

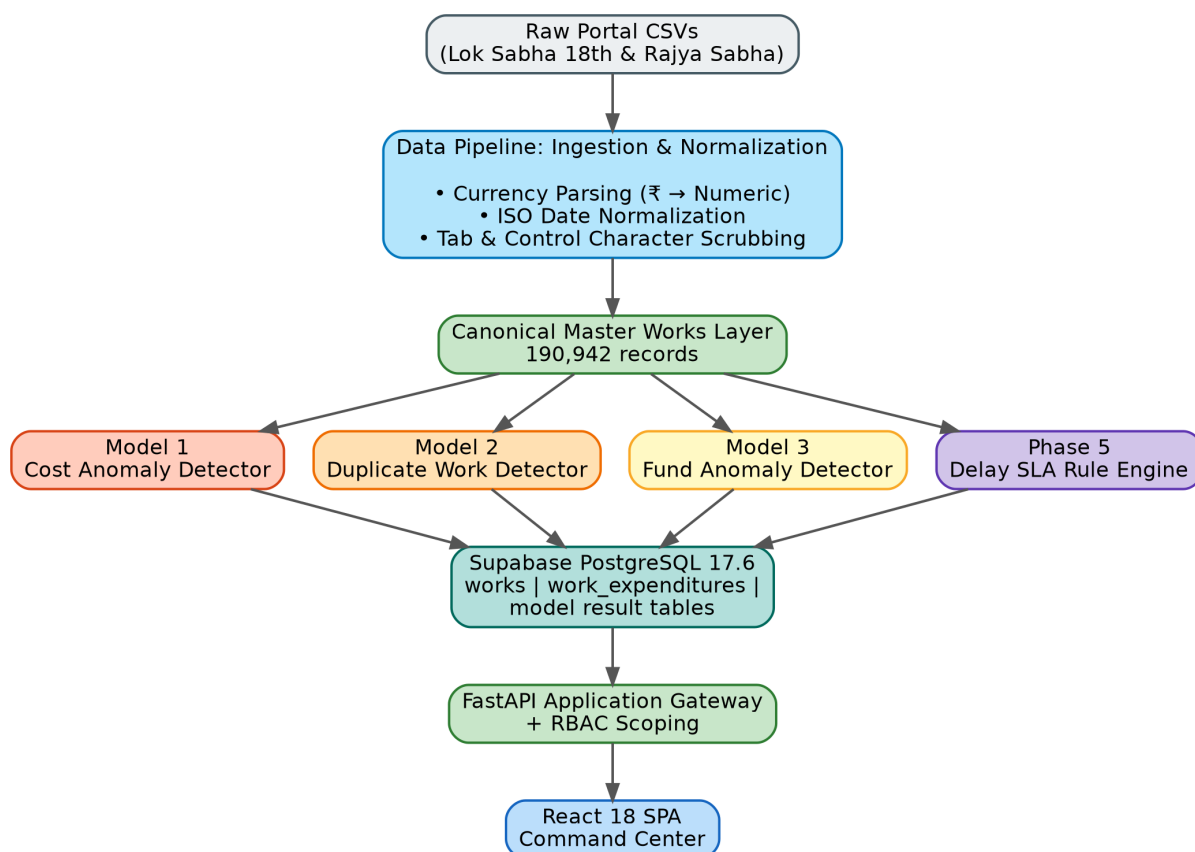


Figure 3: Data lineage from raw portal dumps to the command-center UI

Dataset Statistics & Coverage

Statistic	Value
Total Master Works	190,942
Total Expenditure Vouchers	109,311
States / Union Territories Covered	36
Districts Covered	773
Total Sanctioned Outlay	₹10,211.49 Crore
Total Disbursed Capital	₹10,166.10 Crore
Overall Fund Utilization Rate	99.56%

Module 4: Live Scraper Engine Deep Dive

Architecture (scraper/)

The scraper package is a resilient, asynchronous crawling system designed to extract live data from the official MPLADS portal.

Key Components

1. **Async Spider** (`scraper/spider.py`) — asynchronously requests portal pages, managing connection pooling and retries.
2. **HTML & JSON Parsers** (`scraper/parsers/`) — dedicated parsing modules for works, financial vouchers, and MP profiles (`works.py`, `financials.py`, `mp.py`).
3. **SHA-256 Change Detection** (`scraper/change_detection.py`) — computes SHA-256 hashes of incoming responses to detect whether a record is NEW, UPDATED, or UNCHANGED.
4. **Ingestion Logging** (`database/models.py`) — persists execution metadata across four database tables:
 - `source_snapshots`: raw HTML/JSON response hashes and status codes.
 - `ingestion_runs`: execution start/end timestamps, duration, and record counts.
 - `ingestion_changes`: field-level diff audit logs (old value vs. new value).
 - `scraper_errors`: error stage, type, and stacktrace details.

Change Detection Workflow

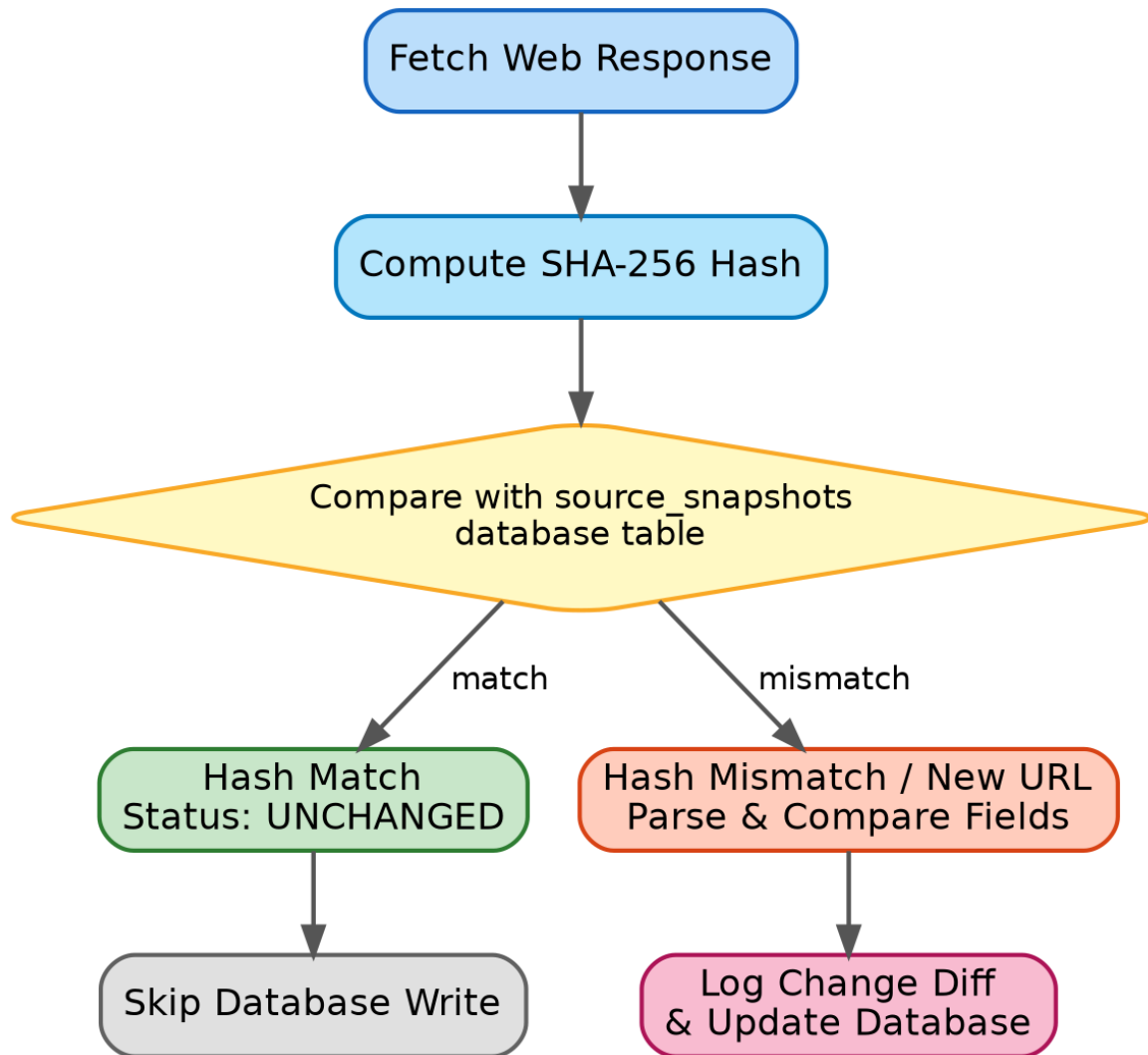


Figure 4: SHA-256 based change detection decision flow

Module 5: Database Architecture & ER Design

Dual-Database & Analytical Parquet Architecture

1. Production Relational Core — Supabase PostgreSQL 17.6

- Role: primary cloud database hosting live web applications.
- Security: Row Level Security (RLS) enabled on all 7 core tables.
- Connection: pooled connection via SQLAlchemy with SSL.
- Integrity: enforces 0 orphaned foreign keys across all analytical result tables.

2. Local Fallback Database — SQLite (mplads_master.db)

Lightweight offline development and testing fallback, automatically configured by database/connection.py when USE_LOCAL_SQLITE=true.

3. Vectorized Analytical Storage — Apache Parquet (data/model_outputs/)

High-speed columnar files used by Python ML libraries (Pandas / Polars / PyArrow) for offline training and vectorized feature computation.

Core Database Tables

Table	Purpose
works	Master catalog of 190,942 works
cost_anomaly_results	Model 1 peer Isolation Forest scores and severities
duplicate_work_results	Model 2 candidate duplicate pairs and structural scores
fund_expenditure_results	Model 3 expenditure anomaly scores and HHI ratios
delay_results	Phase 5 statutory SLA delay scores and primary delay types
work_expenditures	109,311 payment vouchers linked to works via foreign key
users	Stakeholder accounts and jurisdictional scope definitions

Module 6: Analytical & Machine Learning Core Models

The platform strictly avoids artificial risk score averaging. Each model evaluates a distinct compliance dimension.

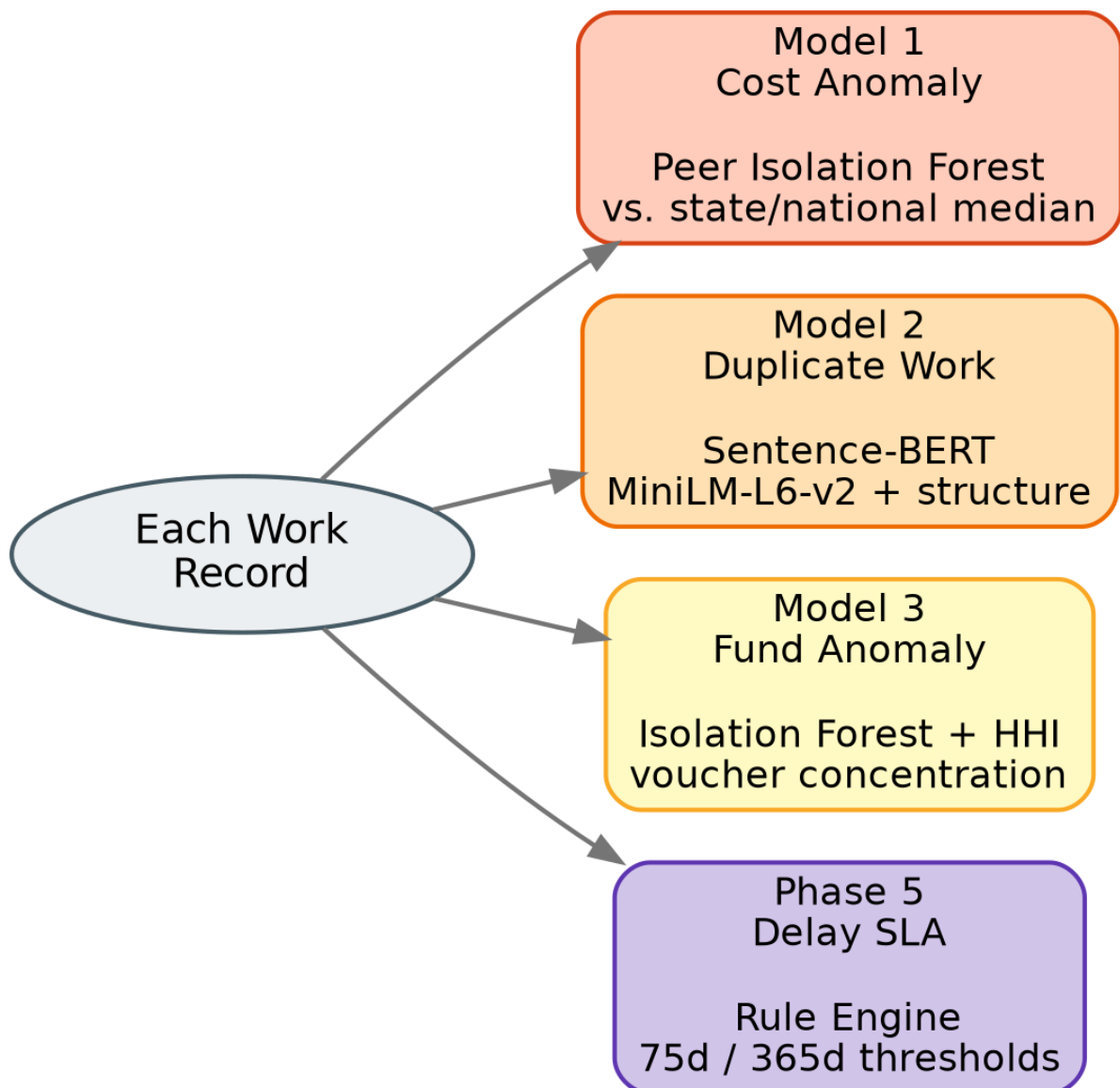


Figure 5: Every work is scored independently by all 4 models

Model 1: Cost Anomaly Detector (ml_models/cost_anomaly/)

- **Purpose:** evaluates cost estimate reasonableness at sanction time.
- **Zero-Leakage:** uses 0 post-sanction features.
- **Hierarchical Peer Grouping:**
 1. STATE_WORK_TYPE (primary, >95% of works)
 2. NATIONAL_WORK_TYPE (fallback when peer count < 15)
 3. NATIONAL_OVERALL (emergency fallback when peer count < 15)
- **Algorithm:** peer-trained Isolation Forest (n_estimators=100, contamination=0.01); raw output mapped to [0,1] via calibrated sigmoid transformation.

Model 2: Duplicate Work Detector (ml_models/duplicate_work/)

- **Purpose:** identifies potential duplicate project sanctions.
- **Candidate Blocking:** 90-day sanction date window within identical state and category (reduces pairwise checks from 4.88 billion to 2.02 million).
- **Embedding Transformer:** sentence-transformers/all-MiniLM-L6-v2 (384-dimensional dense vectors).
- **Scoring Formula:**
$$\text{duplicate_score} = 0.65 \times \text{semantic_similarity} + 0.35 \times \text{structural_score}$$
where the structural score combines amount similarity, date proximity, same-MP, and same-constituency flags.

Model 3: Fund & Expenditure Anomaly Detector (ml_models/fund_e/)

- **Purpose:** monitors payment concentration, velocity, and status mismatches.
- **Cohorts:**
 - Active Spenders (disbursed > 0): Isolation Forest trained on 6 features including the Herfindahl-Hirschman Index (HHI) voucher concentration.
 - Zero-Disbursement (disbursed = 0): deterministic rules flagging dormant sanctions (>365 days old) and status-expenditure mismatches (marked complete but 0 vouchers).

Phase 5: Delay & Statutory SLA Rule Engine (rule_engines/delay/)

- **Purpose:** enforces statutory timelines mandated by Para 3.12 of the MPLADS Guidelines.
- **Track 1:** Recommendation-to-Sanction SLA (75-day limit).
- **Track 2:** Sanction-to-Completion SLA (365-day limit).
- **Track 3:** Open Work Aging, evaluated against reference date 2026-09-05.

Module 7: Feature Engineering Registry

Pipeline (feature_engineering/)

The feature engineering registry converts raw portal records into clean analytical matrices.

Core Feature Transformations

1. **Log Transformations** — `sanction_amount_log`, `log_disbursed_amount`, `days_to_first_disbursement_log` to eliminate monetary skewness.
2. **IQR Deviations** — cost ratio vs. peer group median and IQR deviation metrics.
3. **Herfindahl-Hirschman Index (HHI)** — measures voucher disbursement concentration:
$$HHI = \sum (\text{voucher_amount}_i / \text{total_disbursed})^2$$

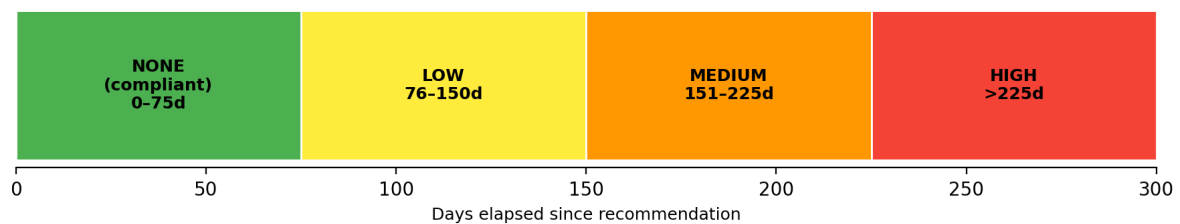
A value of 1.0 indicates a lump-sum single release; a value below 0.2 indicates gradual, phased tranches.
4. **Temporal Lag Vectors** — elapsed days between recommendation, sanction, first voucher, last voucher, and completion.

Module 8: Statutory SLA Rule Engine

Rule Engine (`rule_engines/delay/`)

Statutory compliance in government schemes is a legal mandate, requiring a deterministic rule engine rather than a probabilistic ML model.

Sanction SLA — Para 3.12 (75-Day Legal Limit)



Execution SLA (365-Day Completion Window)

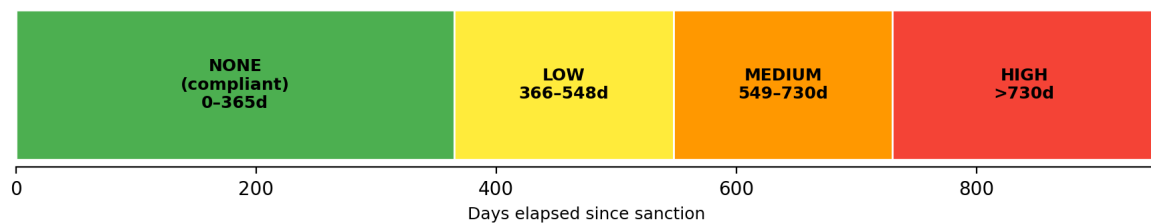


Figure 6: Sanction and execution SLA severity bands

Open Work Aging

Evaluates active elapsed days from the sanction date to the fixed reference date 2026-09-05.

Module 9: FastAPI Backend Application Gateway

Architecture (api/)

The backend gateway acts as the secure intermediary between governance clients and database storage.

Core Configuration

- **Framework:** FastAPI (fastapi $\geq$ 0.115.0) on the Uvicorn ASGI server.
- **Middleware:** CORS middleware, SlowAPI rate limiter (5 requests/minute on login), and a process-time diagnostic header (X-Process-Time-Ms).
- **SPA Routing:** mounts /videos and /assets, serving frontend/dist/index.html for unknown client-side routes.

Primary Endpoint Categories

Route Prefix	Purpose
/api/v1/health	Connection check and latencies
/api/v1/auth/*	Authentication, token generation, user profiles, provisioning
/api/v1/works/*	Work list catalog and single-work dossiers
/api/v1/analytics/*	Model-specific detections and district/MP summaries
/api/v1/chat/*	Subho AI chatbot query endpoints

Module 10: Authentication, RBAC & Security Architecture

Cryptographic Security Stack

- **JWT Standard:** HMAC-SHA256 (HS256) signed tokens with a 60-minute expiration.
- **Password Hashing:** bcrypt with a 12-round cost factor.

Timing Attack Defense

To prevent username enumeration via response timing, `api/auth/security.py` executes a pre-computed 12-round dummy bcrypt hash check when an invalid email is submitted. This ensures uniform response times (~90ms) for all login attempts, whether or not the account exists.

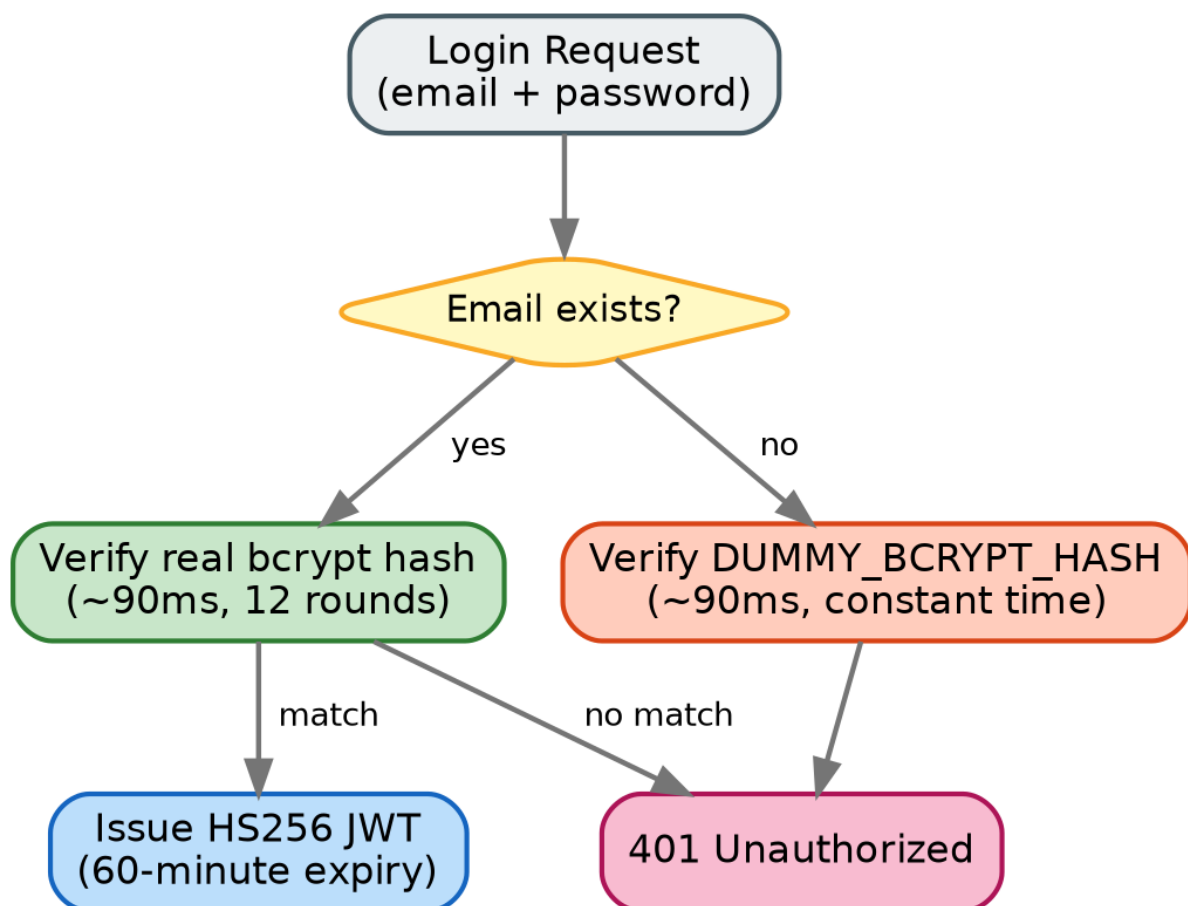


Figure 7: Login flow with constant-time timing-attack defense

Server-Side RBAC & Jurisdictional Scoping

Jurisdictional access control is enforced at the database query level via predicate injection — each role sees a progressively narrower slice of the data:

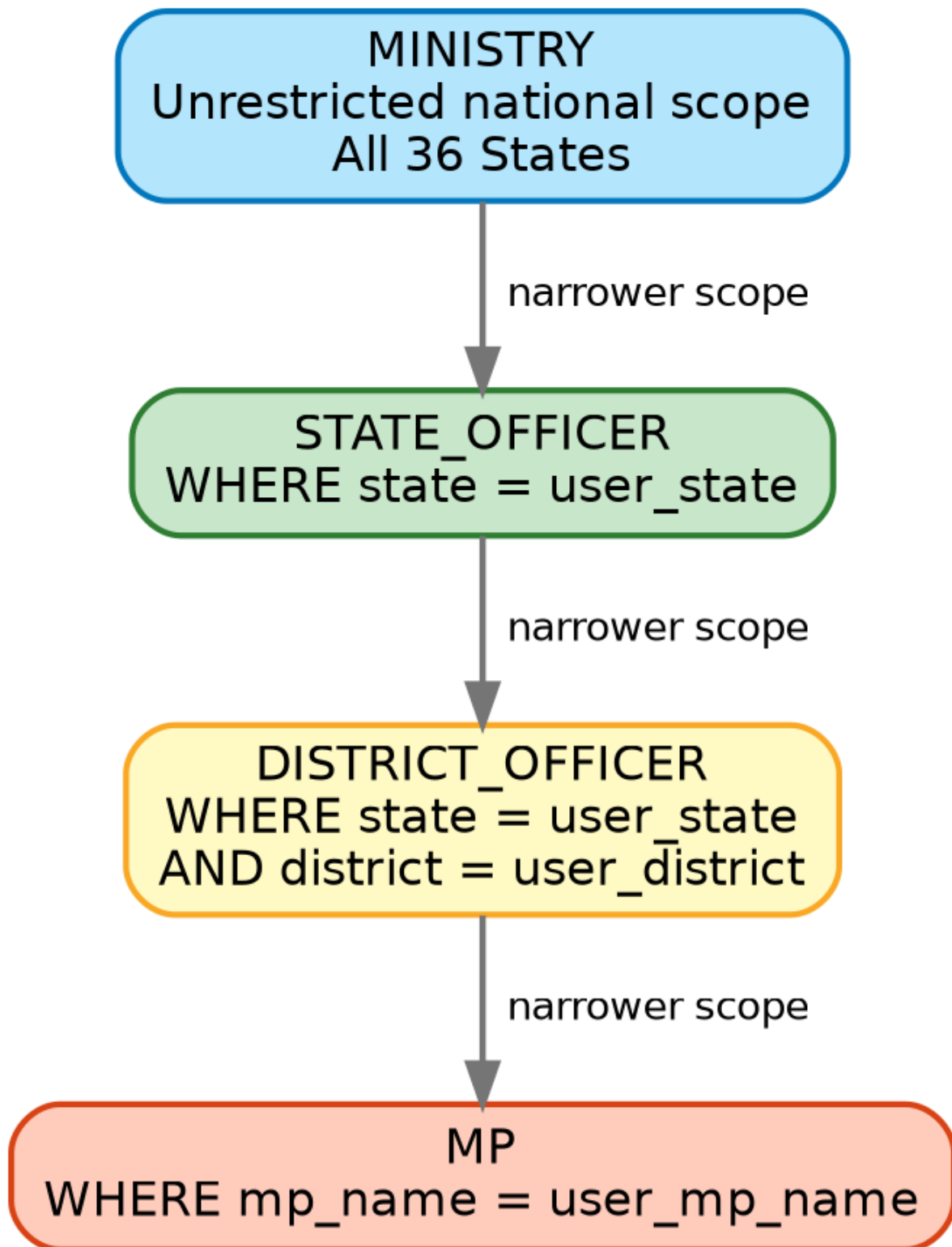


Figure 8: RBAC jurisdiction hierarchy, from national to constituency scope

The composite (state, district) scoping for District Officers resolves the fact that 75 district names are duplicated across different Indian states.

IDOR Protection Strategy

- **List Endpoints:** scoped queries return empty result sets for out-of-jurisdiction filters.
- **Detail Endpoints** (/works/{id}): return 404 Not Found if the work ID does not exist, and 403 Forbidden if the work ID exists but falls outside the caller's jurisdiction.

Module 11: Frontend Command-Center UI/UX

React 18 Single-Page Application (frontend/)

Built with React 18, Vite, TypeScript, Tailwind CSS, Recharts, and Lucide Icons.

Key Pages & Dashboards

Page	Purpose
LandingPage.tsx	Public landing portal with hero background, KPIs, and public chatbot
Login.tsx	Authenticated gateway with a rate-limited credential form
Dashboard.tsx	Executive command center with national metrics and trend charts
WorksRegistry.tsx	Filterable table of all 190,942 works with status badges
WorkDetail.tsx	Single-work dossier showing independent profiles for all 4 models
CostAnomalies.tsx, DuplicateWorks.tsx, FundAnomalies.tsx, StatutoryDelays.tsx	Dedicated per-model risk dashboards
MPSummary.tsx, DistrictSummary.tsx	Jurisdictional portfolio pages for MPs and District Magistrates

Module 12: Subho AI Governance Chatbot

Architecture (`api/routers/chat.py`)

Subho AI is an interactive conversational assistant for platform users.

- **LLM Engine:** Groq API caller with a model fallback chain (llama-3.3-70b-versatile, groq/compound, qwen/qwen3.6-27b).
- **Prompt Injection Defense:** regex pattern matching against prompt override attempts.
- **Output Redaction:** regex filter redacting API keys, database connection strings, and JWT secrets before returning any response.
- **Context Injection:** server-side injection of authorized jurisdictional metrics into the LLM system prompt based on the user's role (PUBLIC, MP, PARLIAMENT, ORGANIZATION, AGENCY).

Module 13: Automated Test Suite & QA

Architecture (tests/)

The platform contains a test suite of **117 automated tests** across 15 test files with a **100% pass rate**.

Test File	Tests	Coverage
test_api.py	15	Core REST API endpoints and dossiers
test_auth_rbac.py	19	Login, timing defense, JWT, 4 RBAC scopes
test_database_ingestior	6	DB connections, schemas, FK integrity
test_delay_rules.py	7	SLA thresholds and open work aging
test_feature_engineerir	10	Canonical building, zero leakage, blocking
test_model1_cost_anomal	6	Peer group Isolation Forest scoring
test_model2_duplicate_v	6	Sentence transformer similarity scoring
test_model3_fund_expenc	6	Cohort Isolation Forest and HHI
test_pipeline.py	10	Currency, date, and string cleaners
test_scraper.py	4	Spider requests and change detection
test_subho_chatbot.py	3	Chatbot injection defenses and fallbacks
test_trend_api.py & test_trend_rollups.py	15	Quarterly trend analytics

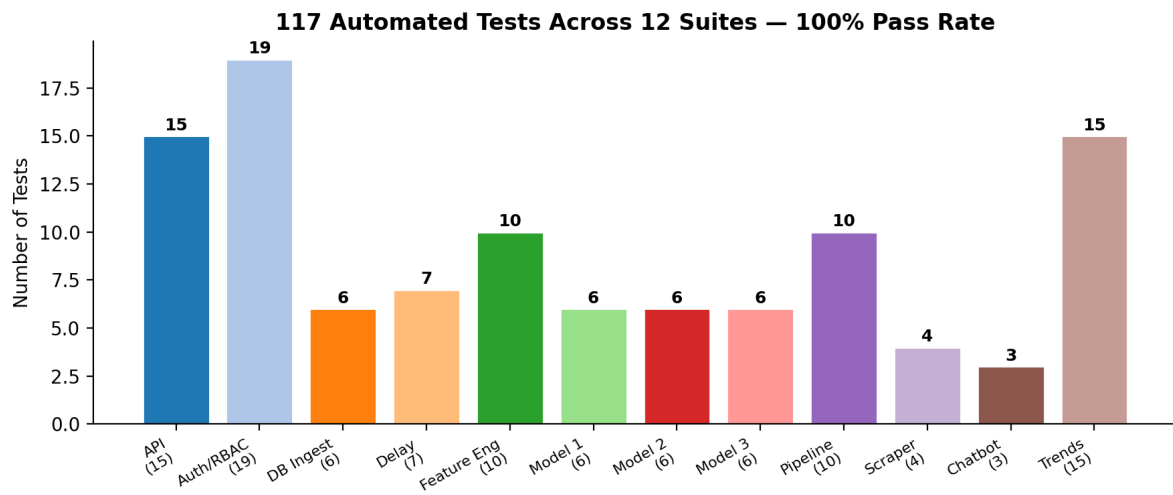


Figure 9: Test distribution across all 12 automated suites

Module 14: Performance Optimization & Deployment

Performance Optimizations

1. **Database Indexing** — B-Tree composite indexes on (state, district), (work_id_1, work_id_2), work_status, and severity.
2. **Columnar Parquet Processing** — offline model scoring uses Apache Parquet for fast, memory-mapped, vectorized Pandas operations.
3. **Single-File SPA Bundle** — the production frontend bundle compiles to a single frontend/dist/index.html (~1,003 kB), served directly by FastAPI.

Deployment Workflow

- **Production Backend:** FastAPI/Uvicorn hosted on a cloud VM (e.g. AWS EC2, Render), connected to Supabase PostgreSQL over SSL.
- **Environment Isolation:** .env file isolated on the server; .env.example committed to the repository as a template.

Module 15: Complete Repository File Map

```
MPLADS-AI-Command-Center/
|-- .env.example          # Environment variables template
|-- .gitattributes        # Standard git line endings config
|-- .gitignore            # Exclusion rules for secrets, DB binaries, caches
|-- README.md             # Primary repository documentation
|-- requirements.txt       # Python dependency specifications
|-- PROJECT_STRUCTURE.md  # Workspace directory breakdown

|-- api/                  # FastAPI Backend Gateway
| |-- config.py, dependencies.py, main.py
| |-- auth/               # Dependencies, limiter, scoping, security
| |-- routers/            # 11 REST controllers
| +-- schemas/            # Pydantic v2 data models

|-- database/             # Relational Core
| +-- connection.py, models.py, schema.sql, populate_sqlite.py,
|    populate_supabase_direct.py

|-- data_pipeline/        # Ingestion & Normalization
| +-- amounts.py, cleaners.py, dates.py, geography.py, pipeline.py

|-- feature_engineering/  # Feature Matrix Extraction
| +-- canonical.py, duplicate_candidates.py, expenditure_features.py,
|    work_features.py

|-- ml_models/            # ML Detection Package
| +-- cost_anomaly/, duplicate_work/, fund_expenditure_anomaly/

|-- rule_engines/         # Statutory Rule Engine
| +-- delay/              # SLA delay evaluation

|-- scraper/              # Live Crawler Engine
| +-- spider.py, client.py, change_detection.py, parsers/

|-- scripts/              # Production Orchestration
| +-- run_full_ml_pipeline.py, build_canonical_dataset.py,
|    verify_full_pipeline.py, security_audit.py

|-- tests/                # 117 Automated Tests
| +-- test_api.py, test_auth_rbac.py, test_delay_rules.py, etc.

|-- frontend/             # React 18 SPA Application
| +-- package.json, vite.config.ts, src/, public/
```

```
+-- docs/                # Documentation & Knowledge Base
  +-- knowledge_base/    # 20 modular technical knowledge base documents
```

Module 16: Architecture & Data Flow Diagrams

High-Level System Flow

This is the same layered architecture from Module 2, shown again here alongside the database relationships for quick reference:

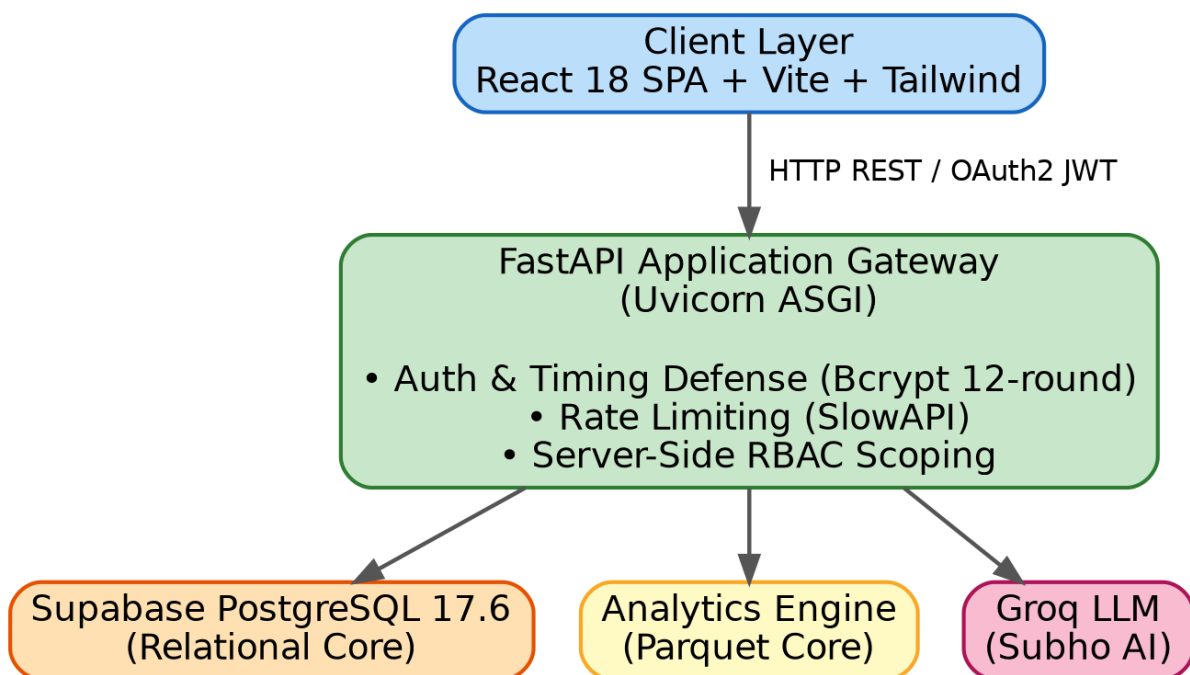


Figure 10: High-level layered architecture (repeated for reference)

Database Relationship Overview

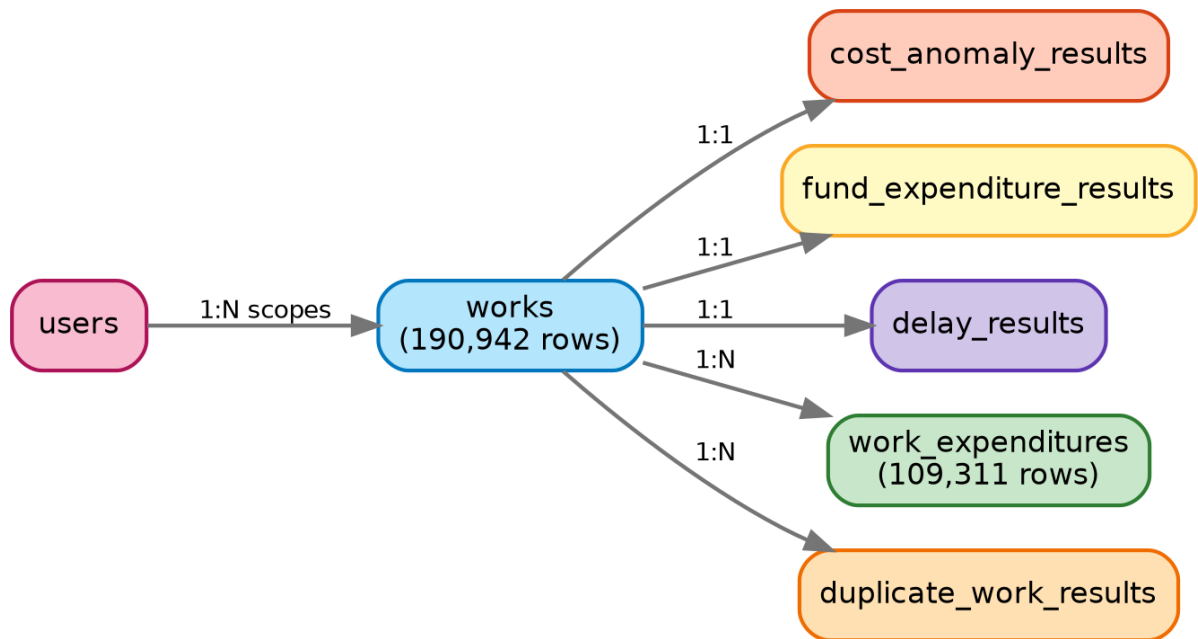


Figure 11: Entity relationships between works, results, and users

Module 17: Viva Questions & Expert Answers

Q1: Why split anomaly detection into 4 separate models instead of one composite risk score?

Short Answer: To preserve signal fidelity. Cost overruns, duplicate billing, dormant funds, and statutory delays are fundamentally different risk dimensions.

Detailed Answer: A composite score averages distinct signals. An urgently completed hospital could have zero delay and valid vouchers but an inflated cost estimate. Averaging these scores would mask the cost anomaly. Keeping models independent gives vigilance officers precise, actionable diagnostic reasons for each domain.

Follow-up (“Why?”): Composite scores create false negatives by diluting extreme single-dimension anomalies.

Q2: How does Model 1 guarantee zero data leakage?

Short Answer: It evaluates works strictly using features available at sanction time.

Detailed Answer: Model 1 uses only `sanction_amount`, `work_type`, and `state`. It strictly excludes post-sanction features like completion dates, voucher counts, or disbursement amounts.

Q3: How do you prevent username enumeration and timing attacks on login?

Short Answer: By executing a constant-time dummy bcrypt hash calculation when an email is not found.

Detailed Answer: Standard backends return immediately if an email doesn't exist (~2ms) but execute bcrypt (~90ms) if it does. Attackers can measure this time difference to harvest valid emails. This system runs a dummy bcrypt hash when the email is missing, forcing every request to take ~90ms regardless of validity.

Q4: Why is District Officer scoping based on (state, district) tuples rather than district names alone?

Short Answer: Because district names in India are not unique across states.

Detailed Answer: 75 district names exist in multiple states (e.g. Bilaspur exists in both Chhattisgarh and Himachal Pradesh). Scoping by district name alone would leak data across state boundaries, so the system enforces composite tuple scoping of `(assigned_state, assigned_district)`.

Module 18: Technology Selection Justifications

Technology Chosen	Alternative Considered	Why Chosen	Why Alternative Was Rejected
FastAPI	Django / Flask	High-performance async ASGI, automatic OpenAPI docs, strict Pydantic validation	Django is too heavy for a REST API; Flask lacks native async and Pydantic integration
Supabase PostgreSQL	MongoDB	Strict relational schema, foreign key constraints, native SQL joins	MongoDB lacks ACID joins needed across works and expenditure tables
Isolation Forest	Random Forest / XGBoost	Unsupervised anomaly detection; no labeled fraud data required	Supervised models need labeled fraud datasets that don't exist in government dumps
Sentence-BERT (all-MiniLM-L6-v2)	TF-IDF / Cosine	Captures semantic context (e.g. "Water Tank" vs "Storage Reservoir")	TF-IDF relies on exact word matches and misses semantic duplicates
Rule Engine (Delay)	Machine Learning	Statutory SLAs are codified legal rules requiring deterministic evaluation	ML adds unnecessary uncertainty to codified legal timelines
Bcrypt (12 rounds)	Plain SHA-256	Slow password hashing resistant to GPU brute-force attacks	SHA-256 is too fast, enabling fast offline dictionary attacks

Module 19: Evaluator & Reviewer Q&A

What makes this project innovative compared to a standard government dashboard?

“Traditional dashboards only display static totals. The MPLADS AI Command Center actively detects risk: it benchmarks cost estimates against peer medians, matches duplicate work candidates using semantic NLP, flags voucher concentration anomalies, and enforces statutory legal deadlines with automated explainability.”

How does the system handle false positives in Duplicate Work Detection?

“In municipal governance, legitimate phased works (e.g. Road Phase 1 and Road Phase 2) share similar language. The duplicate matcher flags candidate pairs for human review rather than automatically deleting records. Vigilance officers receive ranked audit queues with detailed explanation vectors.”

How scalable is the architecture?

“The backend is stateless and horizontally scalable via FastAPI/Uvicorn. PostgreSQL queries are optimized with composite B-Tree indexes. Dense embeddings are pre-cached, and candidate blocking windows reduce duplicate comparisons by over 99.9%.”

Module 20: One-Page Master Reference

- **Project Name:** MPLADS AI Command Center (Governance & Analytics Platform)
- **Core Scope:** 190,942 Master Works · 109,311 Expenditure Vouchers · ₹10,211.49 Cr Sanctioned · 773 Districts · 36 States
- **Stack:** React 18 SPA + Vite + Tailwind · FastAPI + Async Uvicorn · Supabase PostgreSQL 17.6 · PyArrow & Apache Parquet

The 4 Analytical Models

1. **Cost Anomaly** — Hierarchical Peer Isolation Forest (n_estimators=100, sigmoid-calibrated)
2. **Duplicate Work** — all-MiniLM-L6-v2 dense embeddings + structural proximity + 90-day candidate blocking
3. **Fund Anomaly** — Active-spend Isolation Forest + HHI concentration + status mismatch rules
4. **Delay SLA** — Rule engine enforcing 75-day sanction and 365-day completion SLAs against 2026-09-05

Security: OAuth2 JWT (HS256) · Bcrypt 12 rounds · Dummy-hash timing attack defense · 4 RBAC tiers (MINISTRY, STATE_OFFICER, DISTRICT_OFFICER, MP) · Composite district scoping (state, district)

Validation: 117 / 117 automated tests passing (100% pass rate) · 0 critical vulnerabilities identified

Closing Note

This document consolidates the complete technical scope of the MPLADS AI Command Center for easy sharing — covering the problem it solves, how the system is built end-to-end, the reasoning behind each technology choice, and the safeguards built into it. It is intended as a single reference that can be read start to finish or used as a lookup guide by module.